P21 Chair Guide and Implementation-Ready Specification for the Zhao–Cui Fixed-Branch TT Filter

BayesFilter P21 Technical Note

2026-06-02

Abstract

This note is an additive companion to P20. It does not summarize, shorten, replace, or regenerate P20. P20 remains the complete integrated annotated companion to Zhao–Cui and the fixed-branch gradient derivation. P21 adds a guided reading layer for a former chemistry academic chair and an implementation-ready mathematical specification for a later minimal fixed-branch Python implementation. No executable code is produced here.

Contents

1 Non-Replacement Contract

P20 is the accepted integrated companion. P21 has only two jobs:

$$\text{P21} = \text{chair teaching layer} + \text{implementation-ready mathematical specification.} \tag{P21-0}$$

It is not a substitute for P20:

$$\text{P21} \neq \text{shortened P20}, \qquad \text{P21} \neq \text{executable implementation.} \tag{P21-1}$$

Every formula below points back to the P20 fixed-branch equations when possible. The purpose is to slow down the most difficult parts: mass contractions, carried-filter derivatives, fixed solves, and same-branch finite differences.

The reader should keep one distinction in view:

$$\begin{aligned} &\theta = \text{a random parameter coordinate in Zhao–Cui when used,} \\ &\beta = \text{the external differentiated argument.} \end{aligned} \tag{P21-2}$$

The fixed-branch derivative is a derivative with respect to β, after the numerical branch has been fixed.

2 Roadmap: Five Objects

The method is easier to learn if all notation is organized around five objects:

$$q_t, \qquad \phi_t, \qquad C_{t,k}, \qquad \widehat{Z}_t, \qquad \partial_\beta \log \widehat{Z}_t. \tag{P21-3}$$

The table gives the first pass.

Object	Question answered	Stored object	If wrong
q_t	What density must be approximated at time t?	target evaluator at fitting points	wrong scalar and wrong filter
ϕ_t	What square-root function is fitted?	TT evaluator for the square root	negative or biased mass if convention is mixed
$C_{t,k}$	What finite arrays define ϕ_t?	tensor-train cores	no reproducible value path
$\widehat{Z}_t$	What evidence increment is computed?	scalar mass contraction	wrong likelihood contribution
$\partial_\beta \log \widehat{Z}_t$	What derivative is needed?	derivative of same scalar	wrong HMC/MLE gradient contract

2.1 Object 1: The unnormalized density

The first object answers: what is the thing that Bayes' rule asks us to normalize? In ordinary one-step Bayes notation,

$$\text{posterior density} = \frac{\text{likelihood} \times \text{prior}}{\int \text{likelihood} \times \text{prior}}. \tag{P21-3a}$$

The numerator is the unnormalized density. In filtering, the prior at time t is the previous filter propagated through the transition model, so the unnormalized density is

$$q_t(x_t, x_{t-1}; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t). \tag{P21-3b}$$

The implementation-ready question is not merely "what is q_t?" It is: at which points will q_t be evaluated, and in which coordinates? The fixed branch stores a deterministic point table,

$$Z_{\text{fit}} = \begin{bmatrix} z_1^{(1)} & z_2^{(1)} \\ \vdots & \vdots \\ z_1^{(N)} & z_2^{(N)} \end{bmatrix}, \qquad \text{shape}(Z_{\text{fit}}) = (N, 2). \tag{P21-3c}$$

At those points it stores the target vector

$$\widetilde{q}_t^{\text{fit}} = \left(\widetilde{q}_t(z^{(1)}; \beta), \ldots, \widetilde{q}_t(z^{(N)}; \beta) \right)^\top, \qquad \text{shape}(\widetilde{q}_t^{\text{fit}}) = (N,). \tag{P21-3d}$$

If this object is wrong, every downstream quantity is wrong: the fitted square root, the mass, the carried filter, and the derivative.

2.2 Object 2: The square-root approximation

The second object answers: how do we get a nonnegative density from an approximated function? If one directly fits q_t, the approximation can go negative. Zhao–Cui's repair is to fit a square root:

$$\phi_t(z; \beta) \approx e^{c_t/2} \sqrt{\widetilde{q}_t(z; \beta)}. \tag{P21-3e}$$

Then the declared approximate density is

$$\widehat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z). \tag{P21-3f}$$

The implementation-ready object is the fitted value vector

$$y_t^{\text{fit}} = \left(e^{c_t/2} \sqrt{\widetilde{q}_t(z^{(1)}; \beta)}, \ldots, e^{c_t/2} \sqrt{\widetilde{q}_t(z^{(N)}; \beta)} \right)^\top, \qquad \text{shape}(y_t^{\text{fit}}) = (N,). \tag{P21-3g}$$

The derivative vector is

$$\dot{y}_t^{\text{fit}}[j] = e^{c_t/2} \frac{\dot{\widetilde{q}}_t(z^{(j)}; \beta)}{2\sqrt{\widetilde{q}_t(z^{(j)}; \beta)}}. \tag{P21-3h}$$

If this object is wrong, the implementation may still produce a positive density, but it will be the positive density for the wrong scalar.

2.3 Object 3: The tensor-train cores

The third object answers: how is ϕ_t stored without a full grid? For the minimal two-coordinate specification,

$$\phi_t(z_1, z_2; \beta) = \sum_{r=1}^{R} \left[\sum_{\ell=0}^{p-1} C_1[0, \ell, r] b_1(z_1)[\ell] \right] \left[\sum_{m=0}^{p-1} C_2[r, m, 0] b_2(z_2)[m] \right]. \tag{P21-3i}$$

The stored arrays are

$$\text{shape}(C_1) = (1, p, R), \qquad \text{shape}(C_2) = (R, p, 1). \tag{P21-3j}$$

The derivative arrays have the same shapes:

$$\text{shape}(\dot{C}_1) = (1, p, R), \qquad \text{shape}(\dot{C}_2) = (R, p, 1). \tag{P21-3k}$$

If these arrays are copied from β_0 when evaluating $\beta_0 + h$, the finite-difference test no longer checks the derivative of the fitted approximation. The branch freezes the fitting rule, not the core values.

2.4 Object 4: The normalizer

The fourth object answers: what scalar did the filter contribute to the likelihood? With the square-root convention,

$$\widehat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \qquad R_t(\beta) = \int \phi_t(z; \beta)^2 \, \mathrm{d}z. \tag{P21-3l}$$

For the two-coordinate TT this mass is computed by contractions:

$$S_2[r, s] = \sum_{\ell, m} C_2[r, \ell, 0] B_2[\ell, m] C_2[s, m, 0], \tag{P21-3m}$$

$$R_t = \sum_{r, s, \ell, m} C_1[0, \ell, r] B_1[\ell, m] C_1[0, m, s] S_2[r, s]. \tag{P21-3n}$$

The stored scalar objects are

$$S_2 : (R, R), \qquad R_t : \text{scalar}, \qquad \widehat{Z}_t : \text{positive scalar}. \tag{P21-3o}$$

If the normalizer is wrong, the approximate likelihood is wrong even if the filter's shape looks plausible.

2.5 Object 5: The derivative of the log normalizer

The fifth object answers: how does the computed scalar change with the model parameter? The scalar rule is

$$\partial_\beta \log \widehat{Z}_t = \frac{\dot{\widehat{Z}}_t}{\widehat{Z}_t}. \tag{P21-3p}$$

The TT work is to compute $\dot{\widehat{Z}}_t$ for the same scalar. Since c_t and τ_t are frozen,

$$\dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t. \tag{P21-3q}$$

The mass derivative stores

$$\dot{S}_2 : (R, R), \qquad \dot{R}_t : \text{scalar}, \qquad \dot{\widehat{Z}}_t : \text{scalar}. \tag{P21-3r}$$

The filter also needs the derivative of the carried density:

$$\dot{\widehat{p}}_t^{\,\mathrm{ref}}(z_1) = \frac{\dot{a}_t(z_1)}{\widehat{Z}_t} - \frac{a_t(z_1)\dot{\widehat{Z}}_t}{\widehat{Z}_t^2}. \tag{P21-3s}$$

If this carried derivative is missing, the $t+1$ target derivative is incomplete. This is the main hidden failure mode for a superficial implementation.

The exact filtering target in physical variables is, for $t \geq 2$,

$$q_t(x_t, x_{t-1}; \beta) = p_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t). \tag{P21-4}$$

In a fixed-branch implementation one usually works on reference coordinates

$$x_t = a_{t,1} + h_{t,1} z_1, \qquad x_{t-1} = a_{t,2} + h_{t,2} z_2, \qquad z = (z_1, z_2) \in [-1, 1]^2. \tag{P21-5}$$

If the previous filter is stored as a reference-coordinate density $\widehat{p}_{t-1}^{\,\mathrm{ref}}(z_2; \beta)$, then the reference target for $t \geq 2$ is

$$\widetilde{q}_t(z_1, z_2; \beta) = \widehat{p}_{t-1}^{\,\mathrm{ref}}(z_2; \beta) f_\beta(x_t(z_1) \mid x_{t-1}(z_2)) g_\beta(y_t \mid x_t(z_1)) h_{t,1}. \tag{P21-6}$$

The factor is $h_{t,1}$, not $h_{t,1}h_{t,2}$, because $\widehat{p}_{t-1}^{\,\mathrm{ref}}(z_2)\,\mathrm{d}z_2$ already represents the previous filter mass. At $t = 1$, with a physical initial density $p_0(x_0; \beta)$, the reference target is

$$\widetilde{q}_1(z_1, z_0; \beta) = p_0(x_0(z_0); \beta) f_\beta(x_1(z_1) \mid x_0(z_0)) g_\beta(y_1 \mid x_1(z_1)) h_{1,1} h_{1,0}. \tag{P21-7}$$

The fitted square-root values use the P20 shifted convention:

$$y_j(\beta) = \exp(c_t/2) \sqrt{\widetilde{q}_t(z^{(j)}; \beta)}. \tag{P21-8}$$

The tensor train ϕ_t approximates y, and the density used for filtering is

$$\widehat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z), \qquad \int_{[-1,1]^2} \lambda_t(z)\,\mathrm{d}z = 1. \tag{P21-9}$$

The normalizer is

$$\widehat{Z}_t(\beta) = e^{-c_t} \int \phi_t(z; \beta)^2\,\mathrm{d}z + \tau_t. \tag{P21-10}$$

The carried reference filter is

$$\widehat{p}_t^{\,\mathrm{ref}}(z_1; \beta) = \frac{a_t(z_1; \beta)}{\widehat{Z}_t(\beta)}, \qquad a_t(z_1; \beta) = \int_{-1}^{1} \widehat{q}_t(z_1, z_2; \beta)\,\mathrm{d}z_2. \tag{P21-11}$$

Teach-back checkpoint

At this point the chair should be able to say:

stored	$\phi_t,\ C_{t,k},\ \widehat{Z}_t,\ a_t,\ \widehat{p}_t,$
integrated	$\widehat{q}_t(z_1, z_2)$ over one or both coordinates,
differentiated	$\widehat{Z}_t(\beta),\ a_t(z_1; \beta),\ \widehat{p}_t(z_1; \beta),$
frozen	$z^{(j)},\ c_t,\ \tau_t,\ \lambda_t,$ basis, ranks,
breaks claim	changing branch objects between $\beta_0 - h, \beta_0, \beta_0 + h$.

(P21-12)

3 Ladder 1: Normalizer Derivative

Scalar case

Let

$$Z(\beta) = \int q(z;\beta)\,\mathrm{d}z, \qquad Z(\beta) > 0. \tag{P21-13}$$

If differentiation can pass through the integral, then

$$\dot{Z}(\beta) = \int \dot{q}(z;\beta)\,\mathrm{d}z, \qquad \partial_\beta \log Z(\beta) = \frac{\dot{Z}(\beta)}{Z(\beta)}. \tag{P21-14}$$

Two-coordinate case

For $z = (z_1, z_2)$,

$$Z(\beta) = \int_{-1}^{1}\int_{-1}^{1} q(z_1, z_2;\beta)\,\mathrm{d}z_2\,\mathrm{d}z_1, \tag{P21-15}$$

so

$$\dot{Z}(\beta) = \int_{-1}^{1}\int_{-1}^{1} \dot{q}(z_1, z_2;\beta)\,\mathrm{d}z_2\,\mathrm{d}z_1. \tag{P21-16}$$

TT case

In the fixed branch,

$$q(z;\beta) = e^{-c}\phi(z;\beta)^2 + \tau\lambda(z), \tag{P21-17}$$

with c, τ, λ frozen. Therefore

$$\dot{q}(z;\beta) = 2e^{-c}\phi(z;\beta)\dot{\phi}(z;\beta), \tag{P21-18}$$

and

$$\dot{\widehat{Z}}(\beta) = 2e^{-c}\int \phi(z;\beta)\dot{\phi}(z;\beta)\,\mathrm{d}z. \tag{P21-19}$$

P20 computes the same quantity by differentiating mass contractions instead of forming the full integral grid.

Filtering meaning

The scalar used by the fixed branch is

$$\widehat{\ell}_T(\beta; B) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B_t). \tag{P21-20}$$

The branch B_t defines one mathematical function of β. The gradient is

$$\partial_\beta \widehat{\ell}_T(\beta; B) = \sum_{t=1}^{T} \frac{\dot{\widehat{Z}}_t(\beta; B_t)}{\widehat{Z}_t(\beta; B_t)}. \tag{P21-21}$$

Teach-back checkpoint

The chair should be able to say:

$$\text{normalizer derivative} = \text{differentiate the mass before taking } \dot{Z}/Z. \tag{P21-22}$$

What would break it:

$$\widehat{Z}_t(\beta_0 + h) \text{ computed with a different branch than } \widehat{Z}_t(\beta_0). \tag{P21-23}$$

4 Ladder 2: Squared-Density Derivative

Scalar case

If $r(\beta) = \phi(\beta)^2$, then

$$\dot{r}(\beta) = 2\phi(\beta)\dot{\phi}(\beta). \tag{P21-24}$$

This is the elementary reason the derivative of a squared-TT density needs the derivative of the fitted TT cores.

Two-coordinate case

For $\phi(z_1, z_2; \beta)$,

$$R(\beta) = \iint \phi(z_1, z_2; \beta)^2 \, \mathrm{d}z_2 \, \mathrm{d}z_1, \tag{P21-25}$$

and

$$\dot{R}(\beta) = 2 \iint \phi(z_1, z_2; \beta)\dot{\phi}(z_1, z_2; \beta) \, \mathrm{d}z_2 \, \mathrm{d}z_1. \tag{P21-26}$$

TT case

For two coordinates and rank R,

$$\phi(z_1, z_2; \beta) = \sum_{r=1}^{R} u_r(z_1; \beta) v_r(z_2; \beta), \tag{P21-27}$$

where

$$u_r(z_1; \beta) = \sum_{\ell=0}^{p-1} C_1[0, \ell, r] b_1(z_1)[\ell], \qquad v_r(z_2; \beta) = \sum_{m=0}^{p-1} C_2[r, m, 0] b_2(z_2)[m]. \tag{P21-28}$$

Then

$$\dot{\phi} = \sum_{r=1}^{R} \dot{u}_r v_r + \sum_{r=1}^{R} u_r \dot{v}_r. \tag{P21-29}$$

The derivative arrays $\dot{C}_1, \dot{C}_2$ define $\dot{u}, \dot{v}$.

Filtering meaning

Squaring ϕ makes the approximation nonnegative:

$$e^{-c}\phi(z; \beta)^2 + \tau\lambda(z) \geq 0. \tag{P21-30}$$

The derivative does not disturb nonnegativity; it only tells how the mass of this nonnegative density changes with β.

Teach-back checkpoint

The chair should be able to point to the one-line reason for the square:

stored	$\phi_t, \dot{\phi}_t$ through $C_{t,k}, \dot{C}_{t,k}$,
integrated	$e^{-c_t}\phi_t(z)^2 + \tau_t\lambda_t(z)$,
differentiated	$e^{-c_t}\phi_t(z)^2 \mapsto 2e^{-c_t}\phi_t(z)\dot{\phi}_t(z)$,
frozen	$c_t, \tau_t, \lambda_t,$ basis and branch,
breaks claim	differentiating a different fitted square root than the one squared.

(P21-30a)

In words after the math: the square is what buys nonnegativity, and the dot is only a sensitivity of that already-declared nonnegative approximation.

5 Ladder 3: Mass Contraction Derivative

Scalar case

Suppose

$$R(\beta) = c(\beta)^2 M, \tag{P21-31}$$

where M is fixed. Then

$$\dot{R}(\beta) = 2\dot{c}(\beta)c(\beta)M. \tag{P21-32}$$

Two-coordinate rank-one case

If $\phi(z_1, z_2) = u(z_1)v(z_2)$, then

$$R = \left[\int u(z_1)^2 \, \mathrm{d}z_1\right]\left[\int v(z_2)^2 \, \mathrm{d}z_2\right], \tag{P21-33}$$

and

$$\dot{R} = \left[2\int u\dot{u}\right]\left[\int v^2\right] + \left[\int u^2\right]\left[2\int v\dot{v}\right]. \tag{P21-34}$$

Two-coordinate rank-R TT case

Define the one-coordinate basis mass matrices

$$B_k[\ell, m] = \int_{-1}^{1} b_k(z)[\ell] \, b_k(z)[m] \, \mathrm{d}z, \qquad \mathrm{shape}(B_k) = (p, p). \tag{P21-35}$$

For the second core,

$$S_2[r, s] = \sum_{\ell=0}^{p-1}\sum_{m=0}^{p-1} C_2[r, \ell, 0]B_2[\ell, m]C_2[s, m, 0], \qquad \mathrm{shape}(S_2) = (R, R). \tag{P21-36}$$

Then the square integral is

$$R_\phi = \sum_{r,s=1}^{R}\sum_{\ell,m=0}^{p-1} C_1[0, \ell, r]B_1[\ell, m]C_1[0, m, s]S_2[r, s]. \tag{P21-37}$$

Its derivative is the product rule:

$$\begin{aligned}\dot{R}_\phi = &\sum_{r,s,\ell,m} \dot{C}_1[0,\ell,r]B_1[\ell,m]C_1[0,m,s]S_2[r,s] \\ &+ \sum_{r,s,\ell,m} C_1[0,\ell,r]B_1[\ell,m]\dot{C}_1[0,m,s]S_2[r,s] \\ &+ \sum_{r,s,\ell,m} C_1[0,\ell,r]B_1[\ell,m]C_1[0,m,s]\dot{S}_2[r,s].\end{aligned} \tag{P21-38}$$

The derivative of S_2 is

$$\begin{aligned}\dot{S}_2[r,s] = &\sum_{\ell,m} \dot{C}_2[r,\ell,0]B_2[\ell,m]C_2[s,m,0] \\ &+ \sum_{\ell,m} C_2[r,\ell,0]B_2[\ell,m]\dot{C}_2[s,m,0].\end{aligned} \tag{P21-39}$$

Filtering meaning

Mass contraction is simply integration without a full grid. The TT cores store the function; the mass matrices store the one-dimensional integration rules; the contraction order defines the finite-dimensional computation.

Teach-back checkpoint

The chair should be able to say that a mass contraction is an integral written as matrix products:

stored	$C_1, C_2, B_1, B_2, S_2, R_\phi$ and their dotted counterparts,	
integrated	$\phi_t(z_1, z_2)^2$ over z_2 first, then z_1,	
differentiated	$C_2^\top B_2 C_2$ and $C_1^\top B_1 C_1$ by the product rule,	(P21-39a)
frozen	$B_1, B_2,$ coordinate order, rank, basis,	
breaks claim	omitting one of the two $\dot{C}_2$ terms or one of the three $\dot{R}_\phi$ groups.	

This is the part that often looks mysterious. It is not a new probability rule; it is the ordinary integral of a square, evaluated with small mass matrices instead of a large tensor grid.

6 Ladder 4: Fixed Ridge-Solve Derivative

Scalar case

If $ng = d$ with $n \neq 0$, then $g = d/n$. Differentiating $ng = d$ gives

$$\dot{n}\, g + n\, \dot{g} = \dot{d}, \qquad \dot{g} = \frac{\dot{d} - \dot{n}\, g}{n}. \tag{P21-40}$$

Matrix case

For a fixed core update,

$$N_k(\beta) g_k(\beta) = d_k(\beta), \tag{P21-41}$$

where

$$N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{P21-42}$$

Differentiating gives

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k, \tag{P21-43}$$

with

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \qquad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \tag{P21-44}$$

This is P20's fixed-solve derivative written as the coding contract: same left-hand system, new right-hand side.

Two-coordinate design rows

For updating C_1 with C_2 fixed, define

$$V_{j,r} = \sum_{m=0}^{p-1} C_2[r, m, 0] b_2(z_2^{(j)})[m], \qquad \text{shape}(V) = (N, R). \tag{P21-45}$$

The design row is

$$A^{(1)}_{j,(\ell,r)} = b_1(z_1^{(j)})[\ell] V_{j,r}, \qquad \text{shape}(A^{(1)}) = (N, pR). \tag{P21-46}$$

Its derivative is

$$\dot{A}^{(1)}_{j,(\ell,r)} = b_1(z_1^{(j)})[\ell] \dot{V}_{j,r}. \tag{P21-47}$$

For updating C_2 with C_1 fixed, define

$$U_{j,r} = \sum_{\ell=0}^{p-1} C_1[0, \ell, r] b_1(z_1^{(j)})[\ell], \qquad A^{(2)}_{j,(r,m)} = U_{j,r} b_2(z_2^{(j)})[m]. \tag{P21-48}$$

Shapes:

$$\text{shape}(U) = (N, R), \qquad \text{shape}(A^{(2)}) = (N, Rp). \tag{P21-49}$$

Filtering meaning

The fitted cores remain functions of β:

$$C_k(\beta; B) = \text{core returned by the fixed solve rule at } \beta. \tag{P21-50}$$

The branch freezes the rule, not the numerical core values.

Teach-back checkpoint

The chair should be able to teach the fixed solve as the derivative of a linear equation:

stored	$A^{(k)}, N^{(k)}, d^{(k)}, g_k, C_k$ and dotted versions,	
integrated	nothing in this block; it is a fitting solve,	
differentiated	$N^{(k)} g_k = d^{(k)} \mapsto N^{(k)} \dot{g}_k = \dot{d}^{(k)} - \dot{N}^{(k)} g_k$,	(P21-50a)
frozen	$W, \rho, Z_{\text{fit}}, B_{\text{val}}$, sweep order,	
breaks claim	using a different solve system for $\dot{g}_k$ or copying $g_k(\beta_0)$ to $\beta_0 \pm h$.	

After the math: this is why the fixed branch is needed. If the design rule, rank, point set, or sweep order changes, the derivative is no longer the derivative of one declared finite-dimensional solve.

7 Ladder 5: Carried-Filter Quotient Derivative

Scalar case

If

$$p(\beta) = \frac{a(\beta)}{Z(\beta)}, \tag{P21-51}$$

then

$$\dot{p}(\beta) = \frac{\dot{a}(\beta)}{Z(\beta)} - \frac{a(\beta)\dot{Z}(\beta)}{Z(\beta)^2}. \tag{P21-52}$$

Two-coordinate carried numerator

With coordinate order $z = (z_1, z_2) = (z_t, z_{t-1})$,

$$a_t(z_1;\beta) = \int_{-1}^{1} \widehat{q}_t(z_1, z_2;\beta)\,\mathrm{d}z_2. \tag{P21-53}$$

For uniform reference density $\lambda(z_1, z_2) = 1/4$,

$$\int_{-1}^{1} \lambda(z_1, z_2)\,\mathrm{d}z_2 = \frac{1}{2}. \tag{P21-54}$$

Using S_2 from (P21-36),

$$a_t(z_1;\beta) = e^{-c_t} \sum_{r,s,\ell,m} C_1[0,\ell,r] b_1(z_1)[\ell] S_2[r,s] C_1[0,m,s] b_1(z_1)[m] + \frac{\tau_t}{2}. \tag{P21-55}$$

The derivative is

$$\begin{aligned} \dot{a}_t(z_1;\beta) = {} & e^{-c_t} \sum_{r,s,\ell,m} \dot{C}_1[0,\ell,r] b_1(z_1)[\ell] S_2[r,s] C_1[0,m,s] b_1(z_1)[m] \\ & + e^{-c_t} \sum_{r,s,\ell,m} C_1[0,\ell,r] b_1(z_1)[\ell] \dot{S}_2[r,s] C_1[0,m,s] b_1(z_1)[m] \\ & + e^{-c_t} \sum_{r,s,\ell,m} C_1[0,\ell,r] b_1(z_1)[\ell] S_2[r,s] \dot{C}_1[0,m,s] b_1(z_1)[m]. \end{aligned} \tag{P21-56}$$

Then

$$\dot{\widehat{p}}_t^{\,\mathrm{ref}}(z_1;\beta) = \frac{\dot{a}_t(z_1;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(z_1;\beta)\dot{\widehat{Z}}_t(\beta)}{\widehat{Z}_t(\beta)^2}. \tag{P21-57}$$

Filtering meaning

Equation (P21-57) is the derivative object that feeds time $t+1$. Without it, the derivative of the next target omits the dependency of the previous approximate filter on β.

Representation contract for carried one-coordinate objects

The carried filter must be stored as a concrete one-coordinate object, not as an informal phrase. With the normalized Legendre basis from (P21-72), $b_1(z) = (b_1(z)[0], \dots, b_1(z)[p-1])^\top$ and $b_1(z)[0] = 1/\sqrt{2}$. Store the numerator coefficient matrix

$$Q_t[\ell, m] = e^{-c_t} \sum_{r,s=1}^{R} C_1[0, \ell, r] S_2[r, s] C_1[0, m, s] + \tau_t \mathbf{1}_{\ell=0} \mathbf{1}_{m=0}, \qquad \text{shape}(Q_t) = (p, p). \tag{P21-57a}$$

Then

$$a_t(z_1; \beta) = b_1(z_1)^\top Q_t b_1(z_1). \tag{P21-57b}$$

The defensive term is correct because

$$\tau_t b_1(z_1)[0]^2 = \tau_t / 2. \tag{P21-57c}$$

The derivative coefficient matrix is

$$\begin{aligned} \dot{Q}_t[\ell, m] = e^{-c_t} &\sum_{r,s} \dot{C}_1[0, \ell, r] S_2[r, s] C_1[0, m, s] \\ &+ e^{-c_t} \sum_{r,s} C_1[0, \ell, r] \dot{S}_2[r, s] C_1[0, m, s] \\ &+ e^{-c_t} \sum_{r,s} C_1[0, \ell, r] S_2[r, s] \dot{C}_1[0, m, s], \qquad \text{shape}(\dot{Q}_t) = (p, p). \end{aligned} \tag{P21-57d}$$

Store the normalized carried-filter coefficient matrices

$$P_t = \frac{Q_t}{\widehat{Z}_t}, \qquad \dot{P}_t = \frac{\dot{Q}_t}{\widehat{Z}_t} - \frac{Q_t \dot{\widehat{Z}}_t}{\widehat{Z}_t^2}, \qquad \text{shape}(P_t) = \text{shape}(\dot{P}_t) = (p, p). \tag{P21-57e}$$

For any query vector $z^{\text{query}} \in [-1, 1]^M$, build

$$B^{\text{query}}[j, \ell] = b_1(z_j^{\text{query}})[\ell], \qquad \text{shape}(B^{\text{query}}) = (M, p). \tag{P21-57f}$$

The carried evaluator returns vectors

$$\begin{aligned} \widehat{p}_t^{\text{ref}}[j] &= \sum_{\ell,m} B^{\text{query}}[j, \ell] P_t[\ell, m] B^{\text{query}}[j, m], \\ \dot{\widehat{p}}_t^{\text{ref}}[j] &= \sum_{\ell,m} B^{\text{query}}[j, \ell] \dot{P}_t[\ell, m] B^{\text{query}}[j, m], \end{aligned} \qquad \text{shape}(\widehat{p}_t^{\text{ref}}) = \text{shape}(\dot{\widehat{p}}_t^{\text{ref}}) = (M,). \tag{P21-57g}$$

At the next filtering step, $M = N$ and

$$z_j^{\text{query}} = Z_{\text{fit}}[j, 2], \qquad p_j = \widehat{p}_{t-1}^{\text{ref}}[j], \qquad \dot{p}_j = \dot{\widehat{p}}_{t-1}^{\text{ref}}[j]. \tag{P21-57h}$$

Teach-back checkpoint

The chair should be able to say that the carried filter is a normalized one-coordinate quadratic form:

stored	$Q_t, \dot{Q}_t, P_t, \dot{P}_t$ with shape (p, p),
integrated	$\widehat{q}_t(z_1, z_2)$ over z_2,
differentiated	$P_t = Q_t / \widehat{Z}_t \mapsto \dot{P}_t = \dot{Q}_t / \widehat{Z}_t - Q_t \dot{\widehat{Z}}_t / \widehat{Z}_t^2$,
frozen	$b_1,\ c_t, \tau_t,$ branch and query rule,
breaks claim	carrying only values without a declared basis object, or carrying P_t without $\dot{P}_t$.

(P21-57i)

After the math: this contract is what lets the next target derivative know exactly where the previous-filter derivative lives.

8 Ladder 6: Next-Step Target Derivative

For $t \geq 2$, define abbreviations at fitting point j:

$$p_j = \widehat{p}_{t-1}^{\mathrm{ref}}(z_2^{(j)}; \beta), \qquad f_j = f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}), \qquad g_j = g_\beta(y_t \mid x_t^{(j)}). \tag{P21-58}$$

The target value is

$$\widetilde{q}_{t,j} = p_j f_j g_j h_{t,1}. \tag{P21-59}$$

Because the domain is frozen, $\dot{h}_{t,1} = 0$. Hence

$$\dot{\widetilde{q}}_{t,j} = h_{t,1} \left[\dot{p}_j f_j g_j + p_j \dot{f}_j g_j + p_j f_j \dot{g}_j \right]. \tag{P21-60}$$

The square-root fitting value derivative is

$$\dot{y}_j = e^{c_t/2} \frac{\dot{\widetilde{q}}_{t,j}}{2\sqrt{\widetilde{q}_{t,j}}}, \qquad \widetilde{q}_{t,j} > 0. \tag{P21-61}$$

Teach-back checkpoint

The chair should be able to say:

$$\dot{\widetilde{q}}_t = \text{previous-filter derivative} + \text{transition derivative} + \text{observation derivative}. \tag{P21-62}$$

What would break it:

$$\dot{\widehat{p}}_{t-1} \text{ not carried forward.} \tag{P21-63}$$

9 Implementation-Ready Fixed-Branch Specification

This section is written as if a Python implementation were next, but it is not code. It gives formulas, shapes, loop order, and diagnostics.

9.1 Declared constants and shapes

For the minimal specification:

$$T = 2, \qquad D = 2, \qquad z = (z_1, z_2), \qquad R_0 = R_2 = 1, \qquad R_1 = R. \tag{P21-64}$$

Let p be the number of one-dimensional basis functions and N the number of fitting points. The primary arrays have shapes

$$\begin{aligned}
Z_{\mathrm{fit}} &: (N, 2),\\
B_{\mathrm{val}} &: (N, 2, p),\\
w &: (N,),\\
B_1, B_2 &: (p, p),\\
C_1, \dot{C}_1 &: (1, p, R),\\
C_2, \dot{C}_2 &: (R, p, 1),\\
Q_t, \dot{Q}_t &: (p, p),\\
P_t, \dot{P}_t &: (p, p),\\
B^{\mathrm{query}} &: (M, p),\\
\widehat{p}_t^{\mathrm{ref}}, \dot{\widehat{p}}_t^{\mathrm{ref}} &: (M,),\\
A^{(1)} &: (N, pR),\\
A^{(2)} &: (N, Rp),\\
N^{(1)} &: (pR, pR),\\
N^{(2)} &: (Rp, Rp),\\
y, \dot{y} &: (N,).
\end{aligned} \tag{P21-65}$$

The branch manifest is

$$\begin{aligned}
\mathcal{M} = \{&T, D, N, p, R, \ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}, \text{basis},\\
&\text{fitting point rule}, \rho, c_t, \tau_t, \lambda_t, \text{sweep order}, S, \text{seed}\}.
\end{aligned} \tag{P21-66}$$

9.2 Model and derivative formulas

Use

$$x_t = \beta x_{t-1} + \sigma \epsilon_t, \qquad y_t = \sin(x_t) + \eta_t. \tag{P21-67}$$

The transition density is

$$f_\beta(x_t \mid x_{t-1}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_t - \beta x_{t-1})^2}{2\sigma^2}\right], \tag{P21-68}$$

with derivative

$$\partial_\beta f_\beta(x_t \mid x_{t-1}) = f_\beta(x_t \mid x_{t-1}) \frac{(x_t - \beta x_{t-1}) x_{t-1}}{\sigma^2}. \tag{P21-69}$$

The observation density is

$$g_\beta(y_t \mid x_t) = \frac{1}{\sqrt{2\pi r^2}} \exp\left[-\frac{(y_t - \sin x_t)^2}{2r^2}\right]. \tag{P21-70}$$

For this example g_β has no direct β-derivative at fixed x_t:

$$\partial_\beta g_\beta(y_t \mid x_t) = 0. \tag{P21-71}$$

9.3 Basis and fitting points

Use normalized Legendre basis on $[-1, 1]$:

$$b_k(z) = \left(\sqrt{\frac{1}{2}} P_0(z), \sqrt{\frac{3}{2}} P_1(z), \ldots, \sqrt{\frac{2p-1}{2}} P_{p-1}(z)\right)^\top. \tag{P21-72}$$

For the exact Legendre basis,

$$B_k[\ell, m] = \int_{-1}^{1} b_k(z)[\ell] b_k(z)[m] \, \mathrm{d}z = \begin{cases} 1, & \ell = m, \\ 0, & \ell \neq m. \end{cases} \tag{P21-73}$$

The fitting point array is fixed:

$$Z_{\mathrm{fit}}[j, k] \in [-1, 1], \qquad \mathrm{shape}(Z_{\mathrm{fit}}) = (N, 2). \tag{P21-74}$$

The basis value table is

$$B_{\mathrm{val}}[j, k, \ell] = b_k(Z_{\mathrm{fit}}[j, k])[\ell], \qquad \mathrm{shape}(B_{\mathrm{val}}) = (N, 2, p). \tag{P21-75}$$

9.4 One sweep of core fitting

The deterministic initialization is part of the branch. One forward sweep updates C_1 and then C_2; one backward sweep updates C_2 and then C_1. The number of sweeps S is fixed.

Pseudocode block `fit_core_1`:

$$\begin{aligned} &1.\ V_{j,r} \leftarrow \textstyle\sum_m C_2[r, m, 0] B_{\mathrm{val}}[j, 2, m]. \\ &2.\ A^{(1)}_{j,(\ell,r)} \leftarrow B_{\mathrm{val}}[j, 1, \ell] V_{j,r}. \\ &3.\ N^{(1)} \leftarrow (A^{(1)})^\top W A^{(1)} + \rho I. \\ &4.\ d^{(1)} \leftarrow (A^{(1)})^\top W y. \\ &5.\ \mathrm{vec}(C_1) \leftarrow (N^{(1)})^{-1} d^{(1)}. \end{aligned} \tag{P21-76}$$

Pseudocode block `fit_core_2`:

$$\begin{aligned} &1.\ U_{j,r} \leftarrow \textstyle\sum_\ell C_1[0, \ell, r] B_{\mathrm{val}}[j, 1, \ell]. \\ &2.\ A^{(2)}_{j,(r,m)} \leftarrow U_{j,r} B_{\mathrm{val}}[j, 2, m]. \\ &3.\ N^{(2)} \leftarrow (A^{(2)})^\top W A^{(2)} + \rho I. \\ &4.\ d^{(2)} \leftarrow (A^{(2)})^\top W y. \\ &5.\ \mathrm{vec}(C_2) \leftarrow (N^{(2)})^{-1} d^{(2)}. \end{aligned} \tag{P21-77}$$

Derivative pseudocode uses the same block names with dots. For example,

$$\dot{\mathrm{vec}}(C_1) = (N^{(1)})^{-1} \left[\dot{d}^{(1)} - \dot{N}^{(1)} \, \mathrm{vec}(C_1) \right], \tag{P21-78}$$

where $\dot{N}^{(1)}$ and $\dot{d}^{(1)}$ are computed from (P21-44).

9.5 Forward pass pseudocode

At each time t:

$$\begin{aligned} &1.\ \text{Build or reuse } Z_{\mathrm{fit}}, B_{\mathrm{val}}, W, \mathcal{M}. \\ &2.\ \text{Evaluate } \widetilde{q}_{t,j} \text{ using (P21-7) if } t = 1 \text{ and (P21-6) if } t = 2. \\ &3.\ y_j \leftarrow e^{c_t/2} \sqrt{\widetilde{q}_{t,j}}. \\ &4.\ \text{Run the fixed sweep sequence to obtain } C_1, C_2. \\ &5.\ \text{Compute } S_2, R_\phi, \widehat{Z}_t \text{ using (P21-36)–(P21-37) and (P21-10).} \\ &6.\ \text{Compute } Q_t \text{ using (P21-57a), then } P_t \leftarrow Q_t / \widehat{Z}_t. \\ &7.\ \text{Evaluate } \widehat{p}^{\mathrm{ref}}_t \text{ from } B^{\mathrm{query}} P_t B^{\mathrm{query}\top} \text{ using (P21-57g).} \end{aligned} \tag{P21-79}$$

9.6 Derivative pass pseudocode

At each time t:

$$\begin{aligned}
&1.\ \text{Evaluate } \dot{\widetilde{q}}_{t,j} \text{ using model derivatives and carried } \dot{\widehat{p}}_{t-1}.\\
&2.\ \dot{y}_j \leftarrow e^{c_t/2}\dot{\widetilde{q}}_{t,j}/(2\sqrt{\widetilde{q}_{t,j}}).\\
&3.\ \text{Run the fixed derivative sweep using (P21-78).}\\
&4.\ \text{Compute } \dot{S}_2, \dot{R}_\phi, \dot{\widehat{Z}}_t \text{ using (P21-38)–(P21-39).}\\
&5.\ \text{Compute } \dot{Q}_t \text{ using (P21-57d).}\\
&6.\ \dot{P}_t \leftarrow \dot{Q}_t/\widehat{Z}_t - Q_t\dot{\widehat{Z}}_t/\widehat{Z}_t^2.\\
&7.\ \text{Evaluate } \dot{\widehat{p}}_t^{\,\mathrm{ref}} \text{ from } B^{\mathrm{query}}\dot{P}_t B^{\mathrm{query}\top} \text{ using (P21-57g).}\\
&8.\ \partial_\beta\widehat{\ell}_T \leftarrow \partial_\beta\widehat{\ell}_T + \dot{\widehat{Z}}_t/\widehat{Z}_t.
\end{aligned} \tag{P21-80}$$

9.7 Diagnostics

The implementation-ready diagnostics are

$$\begin{array}{ll}
\text{positive target} & \min_j \widetilde{q}_{t,j} > 0,\\
\text{solve conditioning} & \kappa(N^{(k)}) \text{ below declared limit,}\\
\text{mass positivity} & R_\phi \geq 0, \quad \widehat{Z}_t > 0,\\
\text{filter normalization} & \int \widehat{p}_t^{\mathrm{ref}}(z_1)\,\mathrm{d}z_1 = 1,\\
\text{branch identity} & \mathcal{M}(\beta_0 - h) = \mathcal{M}(\beta_0) = \mathcal{M}(\beta_0 + h).
\end{array} \tag{P21-81}$$

10 Finite-Difference Protocol

The same-branch finite-difference protocol is a mathematical contract for a later implementation. It does not report a numerical result.

At base point β_0, freeze the branch manifest:

$$\mathcal{M}_0 = \mathcal{M}(\beta_0). \tag{P21-82}$$

For each $h \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$, evaluate

$$\widehat{\ell}_2(\beta_0 + h; \mathcal{M}_0), \qquad \widehat{\ell}_2(\beta_0 - h; \mathcal{M}_0), \tag{P21-83}$$

where all branch choices are reused but the fitted cores are recomputed:

$$C_k(\beta_0 \pm h; \mathcal{M}_0) = \text{result of the fixed fitting rule at } \beta_0 \pm h. \tag{P21-84}$$

The centered difference is

$$D(h) = \frac{\widehat{\ell}_2(\beta_0 + h; \mathcal{M}_0) - \widehat{\ell}_2(\beta_0 - h; \mathcal{M}_0)}{2h}. \tag{P21-85}$$

Compare with the analytic derivative

$$G = \partial_\beta\widehat{\ell}_2(\beta_0; \mathcal{M}_0), \qquad e(h) = |D(h) - G|, \qquad e_{\mathrm{rel}}(h) = \frac{|D(h) - G|}{1 + |G|}. \tag{P21-86}$$

The later implementation must print or record the report object

$$\begin{aligned}
\mathcal{R}_{\mathrm{FD}} = \{&\widehat{\ell}_2(\beta_0; \mathcal{M}_0),\ G,\ (h_i, D(h_i), e(h_i), e_{\mathrm{rel}}(h_i))_{i=1}^4,\\
&I_-(h_i), I_+(h_i), K_-(h_i), K_+(h_i),\ W_i,\ \text{status}\},
\end{aligned} \tag{P21-86a}$$

where

$$\begin{aligned} I_{\pm}(h_i) &= \mathbf{1}\{\mathcal{M}(\beta_0 \pm h_i) = \mathcal{M}_0\}, \\ K_{\pm}(h_i) &= \mathbf{1}\{C_k(\beta_0 \pm h_i; \mathcal{M}_0) \text{ was recomputed by the fixed solve rule}\}, \\ W_i &= \mathbf{1}\{e(h_i) > e(h_{i+1}) > e(h_{i+2})\}, \qquad i = 1, 2. \end{aligned} \tag{P21-86b}$$

The pass/fail status is

$$\text{status} = \begin{cases} \texttt{FAIL\_BRANCH}, & \min_{i,\pm} I_{\pm}(h_i) = 0, \\ \texttt{FAIL\_COPIED\_CORES}, & \min_{i,\pm} K_{\pm}(h_i) = 0, \\ \texttt{PASS}, & \max(W_1, W_2) = 1, \\ \texttt{INCONCLUSIVE\_NO\_DECREASING\_WINDOW}, & \text{otherwise.} \end{cases} \tag{P21-86c}$$

Expected behavior:

$$e(10^{-2}) > e(10^{-3}) > e(10^{-4}) \quad \text{over at least one stable window,} \tag{P21-87}$$

with possible roundoff deterioration at 10^{-5}. If no decreasing window appears, first check branch identity, then target derivatives, then solve derivatives, then carried-filter derivatives.

11 Why This Is A Plausible High-Dimensional Filtering Method

The core mathematical argument is

$$\begin{aligned} \widehat{q}_t(z; \beta) &= e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z) \geq 0, \\ 0 < \widehat{Z}_t(\beta) &= \int \widehat{q}_t(z; \beta)\, \mathrm{d}z < \infty, \\ \widehat{p}_t^{\,\mathrm{ref}}(z_1; \beta) &= \frac{\int \widehat{q}_t(z_1, z_2; \beta)\, \mathrm{d}z_2}{\widehat{Z}_t(\beta)}. \end{aligned} \tag{P21-88}$$

Therefore

$$\int \widehat{p}_t^{\,\mathrm{ref}}(z_1; \beta)\, \mathrm{d}z_1 = \frac{\iint \widehat{q}_t(z_1, z_2; \beta)\, \mathrm{d}z_2\, \mathrm{d}z_1}{\widehat{Z}_t(\beta)} = 1. \tag{P21-89}$$

This proves that the fixed branch defines a normalized approximate filter. It does not prove that the approximation equals the exact posterior. The plausibility comes from the combination:

$$\begin{aligned} &\text{nonnegative approximation} + \text{computable mass} + \text{recursive marginal} \\ &\quad \Longrightarrow \text{valid normalized approximate filter.} \end{aligned} \tag{P21-90}$$

The remaining scientific question is empirical and numerical: whether TT ranks, domains, basis choices, and preconditioning make this approximation accurate and stable for the target high-dimensional model.

12 What Is Not Claimed

P21 does not claim

exact posterior accuracy,	$\widehat{q}_t = q_t$,
global adaptive differentiability,	$\partial_\beta B(\beta)$ for changing ranks/pivots/domains,
production readiness,	no production BayesFilter code is written,
HMC convergence,	same-scalar gradient is only one requirement,
full Zhao–Cui implementation,	adaptive TT-cross and KR engineering remain gaps.

(P21-91)